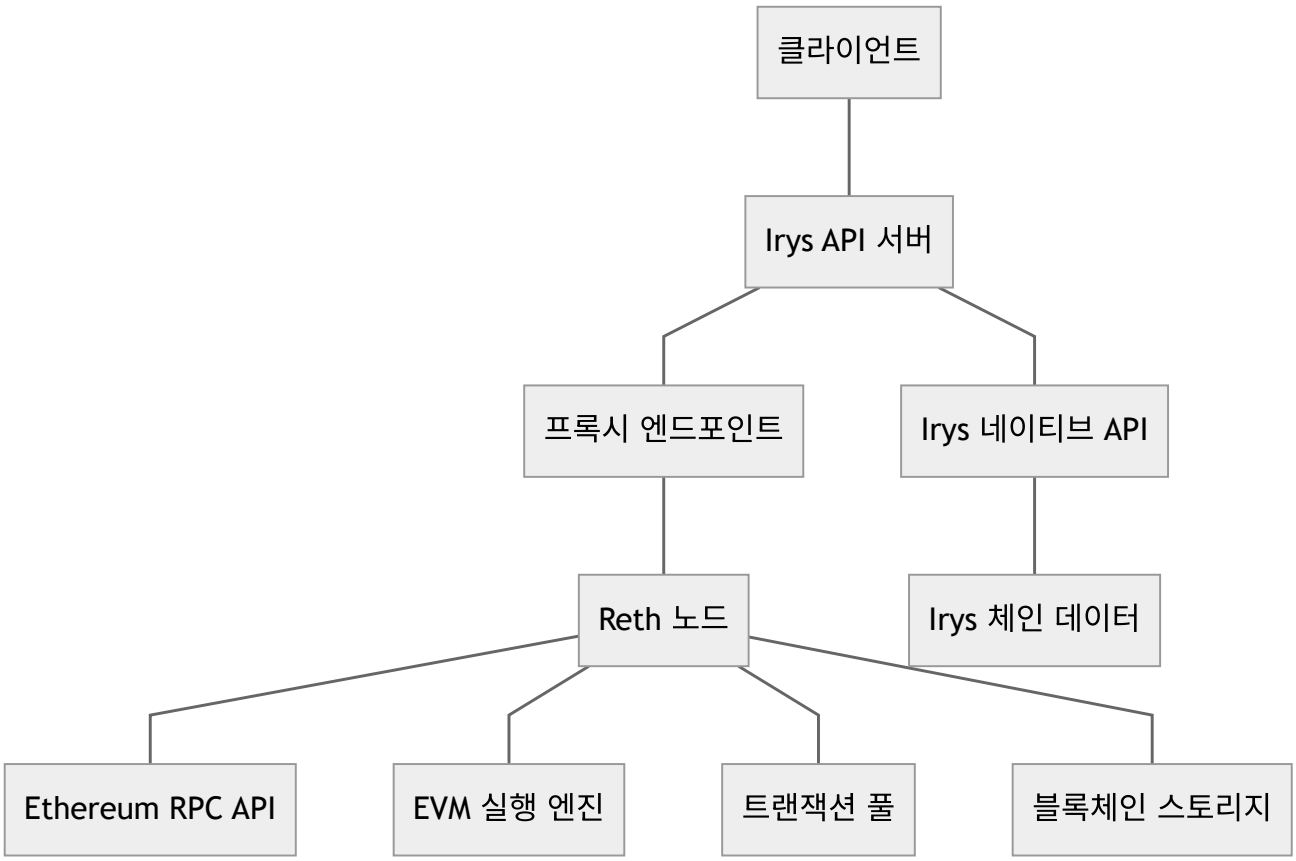
RPC 인터페이스 및 프록시 시스템 문서

개요

Irys 체인은 Ethereum 호환 RPC 인터페이스를 제공하여 기존 Ethereum 도구와 라이브러리와의 호환성을 보장합니다. 이는 Reth 노드를 통해 구현되며, 프록시 시스템을 통해 Ethereum RPC 호출을 처리합니다.

아키텍처 개요



Ethereum RPC 호환성

지원되는 RPC 모듈

Irys 체인은 다음 Ethereum RPC 모듈을 지원합니다:

1. **eth** - 표준 Ethereum JSON-RPC API
2. **debug** - 디버깅 및 트레이싱 API

RPC 설정

```
// Reth 노드 설정에서 RPC 모듈 활성화
reth_config.rpc.http_api = Some(RpcModuleSelection::Selection(HashSet::from([
RethRpcModule::Eth,
RethRpcModule::Debug,
])));
```

프록시 시스템

프록시 엔드포인트

엔드포인트: `ALL /v1/execution-rpc`

모든 HTTP 메서드(GET, POST, PUT, DELETE, HEAD, OPTIONS, PATCH)를 지원하며, 요청을 Reth 노드의 HTTP RPC 서버로 투명하게 전달합니다.

프록시 구현 세부사항

```
pub async fn proxy(
req: HttpRequest,
payload: Payload,
client: Data<Client>,
```

```
state: web::Data<ApiState>,

) -> Result<HttpResponse, ProxyError>
```

주요 기능

1. **요청 전달**

- 클라이언트 요청을 Reth 노드로 투명하게 전달
- HTTP 메서드, 헤더, 본문을 그대로 유지
- 스트리밍 방식으로 대용량 응답 처리

2. **헤더 필터링**

- Hop-by-hop 헤더 제거
- 프록시 관련 헤더 정리
- 응답 헤더 적절히 전달

3. **오류 처리**

- 네트워크 오류 시 502 Bad Gateway 반환
- 파싱 오류 시 400 Bad Request 반환
- 지원하지 않는 메서드 시 405 Method Not Allowed 반환

필터링되는 헤더

다음 hop-by-hop 헤더들은 프록시 과정에서 제거됩니다:

- `connection`
- `keep-alive`
- `proxy-authenticate`
- `proxy-authorization`

- `te`
- `trailers`
- `transfer-encoding`
- `upgrade`

지원되는 Ethereum RPC 메서드

eth 모듈

블록 관련 메서드

- `eth_blockNumber` - 최신 블록 번호 반환
- `eth_getBlockByNumber` - 블록 번호로 블록 정보 조회
- `eth_getBlockByHash` - 블록 해시로 블록 정보 조회
- `eth_getBlockTransactionCountByNumber` - 블록의 트랜잭션 수 조회
- `eth_getBlockTransactionCountByHash` - 블록의 트랜잭션 수 조회

트랜잭션 관련 메서드

- `eth_getTransactionByHash` - 트랜잭션 해시로 트랜잭션 조회
- `eth_getTransactionByBlockHashAndIndex` - 블록 해시와 인덱스로 트랜잭션 조회
- `eth_getTransactionByBlockNumberAndIndex` - 블록 번호와 인덱스로 트랜잭션 조회
- `eth_getTransactionReceipt` - 트랜잭션 영수증 조회
- `eth_sendRawTransaction` - 서명된 트랜잭션 전송
- `eth_pendingTransactions` - 대기 중인 트랜잭션 목록

계정 및 잔액 관련 메서드

- `eth_getBalance` - 계정 잔액 조회

- `eth_getTransactionCount` - 계정의 nonce 조회
- `eth_getCode` - 계정의 바이트코드 조회
- `eth_getStorageAt` - 스토리지 슬롯 값 조회

네트워크 정보 메서드

- `eth_chainId` - 체인 ID 반환
- `eth_gasPrice` - 현재 가스 가격 조회
- `eth_estimateGas` - 가스 사용량 추정
- `eth_feeHistory` - 수수료 히스토리 조회

필터 및 로그 메서드

- `eth_newFilter` - 새 필터 생성
- `eth_newBlockFilter` - 새 블록 필터 생성
- `eth_newPendingTransactionFilter` - 새 대기 트랜잭션 필터 생성
- `eth_uninstallFilter` - 필터 제거
- `eth_getFilterChanges` - 필터 변경사항 조회
- `eth_getFilterLogs` - 필터 로그 조회
- `eth_getLogs` - 로그 조회

호출 메서드

- `eth_call` - 스마트 컨트랙트 함수 호출 (상태 변경 없음)
- `eth_createAccessList` - 액세스 리스트 생성

debug 모듈

트레이싱 메서드

- `debug_traceTransaction` - 트랜잭션 실행 추적
- `debug_traceBlockByNumber` - 블록 실행 추적 (번호 기준)
- `debug_traceBlockByHash` - 블록 실행 추적 (해시 기준)
- `debug_traceCall` - 호출 실행 추적

상태 관련 메서드

- `debug_dumpBlock` - 블록 상태 덤프
- `debug_accountRange` - 계정 범위 조회
- `debug_storageRangeAt` - 스토리지 범위 조회

사용 예시

JSON-RPC 요청 예시

최신 블록 번호 조회

```
curl -X POST http://localhost:8081/v1/execution-rpc \
-H "Content-Type: application/json" \
-d '{
"jsonrpc": "2.0",
"method": "eth_blockNumber",
"params": [],
"id": 1
}'
```

응답:

```
{
"jsonrpc": "2.0",
"id": 1,
"result": "0x1b4"
}
```

블록 정보 조회

```
curl -X POST http://localhost:8081/v1/execution-rpc \
-H "Content-Type: application/json" \
-d '{
"jsonrpc": "2.0",
"method": "eth_getBlockByNumber",
"params": ["latest", true],
"id": 1
}'
```

응답:

```
{
"jsonrpc": "2.0",
"id": 1,
"result": {
"number": "0x1b4",
```

```
"hash": "0x1234567890abcdef...",
"parentHash": "0xfedcba0987654321...",
"timestamp": "0x61bc032c",
"gasLimit": "0x1c9c380",
"gasUsed": "0x5208",
"transactions": [...]
}
}
```

계정 잔액 조회

```
curl -X POST http://localhost:8081/v1/execution-rpc \
-H "Content-Type: application/json" \
-d '{
"jsonrpc": "2.0",
"method": "eth_getBalance",
"params": ["0x742d35Cc6634C0532925a3b8D4C0d886E4C2e7b9", "latest"],
"id": 1
}'
```

트랜잭션 전송

```
curl -X POST http://localhost:8081/v1/execution-rpc \
-H "Content-Type: application/json" \
```

```
-d '{
"jsonrpc": "2.0",
"method": "eth_sendRawTransaction",
"params": ["0xf86c808504a817c800825208..."],
"id": 1
}'
```

Web3.js 사용 예시

```
const Web3 = require('web3');

// Irys 노드에 연결
const web3 = new Web3('http://localhost:8081/v1/execution-rpc');

// 최신 블록 번호 조회
const blockNumber = await web3.eth.getBlockNumber();
console.log('Latest block:', blockNumber);

// 계정 잔액 조회
const balance = await web3.eth.getBalance('0x742d35Cc6634C0532925a3b8D4C0d88
console.log('Balance:', web3.utils.fromWei(balance, 'ether'), 'ETH');

// 트랜잭션 전송
```

```
const tx = {
from: '0x742d35Cc6634C0532925a3b8D4C0d886E4C2e7b9',
to: '0x1234567890123456789012345678901234567890',
value: web3.utils.toWei('1', 'ether'),
gas: 21000
};

const receipt = await web3.eth.sendTransaction(tx);
console.log('Transaction hash:', receipt.transactionHash);
```

ethers.js 사용 예시

```
const { ethers } = require('ethers');

// Irys 노드에 연결
const provider = new ethers.JsonRpcProvider('http://localhost:8081/v1/execut

// 최신 블록 조회
const block = await provider.getBlock('latest');
console.log('Latest block:', block.number);

// 계정 잔액 조회
```

```
const balance = await provider.getBalance('0x742d35Cc6634C0532925a3b8D4C0d88
console.log('Balance:', ethers.formatEther(balance), 'ETH');


// 스마트 컨트랙트 호출
const contract = new ethers.Contract(contractAddress, abi, provider);
const result = await contract.someMethod();
```

Reth 노드 통합

노드 설정

Irys 체인은 Reth 노드를 내장하여 Ethereum 호환성을 제공합니다:

```
// RPC 설정
reth_config.rpc.http = true;
reth_config.rpc.http_api = Some(RpcModuleSelection::Selection(HashSet::from([
RethRpcModule::Eth,
RethRpcModule::Debug,
])));
reth_config.rpc.http_corsdomain = Some("*".to_string());
```

페이로드 빌더 통합

Irys 체인은 커스텀 페이로드 빌더를 사용하여 Irys 트랜잭션과 Ethereum 트랜잭션을 통합합니다:

```
pub  async  fn  build_submit_payload_irys(
&self,
parent: B256,
attributes: PayloadAttributes,
shadow_txs: Vec<EthPooledTransaction>,
) -> eyre::Result<BuiltPayload>
```

섀도우 트랜잭션 시스템

Irys는 섀도우 트랜잭션 시스템을 통해 Irys 네이티브 트랜잭션을 Ethereum 블록에 포함시킵니다:

- **ShadowTxStore**: 섀도우 트랜잭션 저장소
- **DeterministicShadowTxKey**: 결정론적 키 생성
- 페이로드 빌드 시 섀도우 트랜잭션 자동 포함

커스텀 RPC 메서드

Irys 확장 메서드

Irys 체인은 표준 Ethereum RPC 외에도 다음과 같은 커스텀 메서드를 제공할 수 있습니다:

irys_getDataTransaction

Irys 데이터 트랜잭션 정보를 조회합니다.

```
{
"jsonrpc":  "2.0",
"method":  "irys_getDataTransaction",
```

```
"params": ["0x1234567890abcdef..."],
"id": 1
}
```

irys_getChunkByDataRoot

데이터 루트로 청크 정보를 조회합니다.

```
{
"jsonrpc": "2.0",
"method": "irys_getChunkByDataRoot",
"params": ["0x1234567890abcdef...", 0],
"id": 1
}
```

irys_getStoragePrice

데이터 저장 가격을 조회합니다.

```
{
"jsonrpc": "2.0",
"method": "irys_getStoragePrice",
"params": [1, 1024],
"id": 1
}
```

성능 최적화

연결 풀링

프록시 시스템은 HTTP 클라이언트 연결 풀을 사용하여 성능을 최적화합니다:

```
let  client = Client::builder()
.timeout(CLIENT_TIMEOUT)
.read_timeout(CLIENT_TIMEOUT)
.build()
.unwrap();
```

스트리밍 응답

대용량 응답은 스트리밍 방식으로 처리하여 메모리 사용량을 최적화합니다:

```
Ok(client_response.streaming(response))
```

캐싱 전략

- 블록 정보 캐싱
- 트랜잭션 영수증 캐싱
- 계정 상태 캐싱 (적절한 무효화 정책과 함께)

보안 고려사항

요청 검증

- JSON-RPC 요청 형식 검증
- 매개변수 타입 및 범위 검증
- 악의적인 요청 필터링

속도 제한

- IP별 요청 속도 제한
- 메서드별 속도 제한
- 리소스 집약적 메서드에 대한 특별 제한

액세스 제어

- 민감한 debug 메서드에 대한 액세스 제어
- 관리자 전용 메서드 보호
- 네트워크 레벨 액세스 제어

모니터링 및 로깅

메트릭 수집

- RPC 호출 수 및 응답 시간
- 오류율 및 오류 타입
- 리소스 사용량 (CPU, 메모리, 네트워크)

로깅

```
tracing::info!("RPC request: {} {}", method, path);
```

```
tracing::error!("RPC error: {:?}", error);
```

알림

- 높은 오류율 감지
- 응답 시간 임계값 초과
- 리소스 사용량 임계값 초과

문제 해결

일반적인 문제

1. **연결 오류**
- Reth 노드 상태 확인
- 네트워크 연결 확인
- 포트 및 방화벽 설정 확인
2. **응답 지연**
- Reth 노드 성능 확인
- 네트워크 대역폭 확인
- 캐시 설정 최적화
3. **호환성 문제**
- 클라이언트 라이브러리 버전 확인
- RPC 메서드 지원 여부 확인
- 매개변수 형식 확인

디버깅 도구

- RPC 호출 로그 분석
- 네트워크 트래픽 모니터링
- Reth 노드 상태 확인 도구

향후 개발 계획

추가 RPC 모듈

- `net` - 네트워크 정보 API
- `web3` - Web3 유틸리티 API
- `txpool` - 트랜잭션 풀 관리 API

성능 개선

- GraphQL 인터페이스 지원
- WebSocket 기반 실시간 이벤트
- 배치 요청 최적화

보안 강화

- OAuth 2.0 인증 지원
- API 키 기반 인증
- 세밀한 권한 제어